Sistemas Distribuídos - CEFET - MG

Trabalho Prático 2

Transferência de Arquivos Peer-to-Peer

Professora: Michelle Hanne - 2026/2

1 Objetivos

Este projeto consiste na implementação de um sistema elementar de transferência de arquivos utilizando o modelo **Peer-to-Peer** (P2P). O objetivo é que cada par (Peer) possa atuar simultaneamente como cliente e servidor, fragmentando um arquivo de origem e transferindo seus pedaços diretamente para outro par que o solicita.

Destacam-se os seguintes objetivos específicos:

- Compreender e aplicar o modelo de comunicação P2P simétrica.
- Implementar a fragmentação e remontagem de arquivos grandes.
- Gerenciar a comunicação simultânea e bidirecional (cliente e servidor) em um único nó.
- Lidar com a recuperação de dados entre pares conectados.

Além da implementação, você deve testar seu programa, rodando os estudos de casos. Você deve preparar um relatório, com no máximo 5 páginas, com as decisões de projeto e implementação das funcionalidades especificadas, assim como a avaliação dos estudos de caso. O relatório deve conter a URL para o código-fonte da sua implementação. O trabalho pode ser realizado em dupla ou individual. A linguagem de programação adotada na solução é uma decisão de projeto.

2 Requisitos Arquiteturais

O sistema será composto por múltiplos *Peers* (Mínimo de 2 para o teste), onde cada um é um processo que desempenha as seguintes funções:

Peer (Nó):

- **Simetria:** Deve operar simultaneamente como um **Servidor** (ouvindo em uma porta específica para solicitações de blocos) e um **Cliente** (iniciando conexões para solicitar blocos de outros pares).
- **Configuração:** Cada *Peer* deve ser iniciado sabendo seu próprio endereço IP e Porta. A implementação de um Tracker será opcional. O endereço IP e Porta de, pelo menos, um ou mais *Peers* Vizinhos (a lista de vizinhos deve ser predefinida, eliminando a necessidade de um Tracker).
- **Fragmentação:** O *Peer* que possui o arquivo completo (o Seeder inicial) deve dividir o arquivo em blocos de tamanho fixo (ex: 1024 bytes).

Mecanismo de Transferência:

- O Peer que está baixando (o Leecher), ao iniciar, conecta-se a todos os seus vizinhos configurados..

- **O** Leecher deve solicitar blocos do arquivo de seus vizinhos até ter o arquivo completo.
- **T**ornando-se um Seeder: À medida que um Leecher recebe e armazena um bloco com sucesso, ele deve estar pronto para servir (compartilhar) esse mesmo bloco com qualquer outro Peer que o solicite.

3 Implementação e Testes

A implementação da solução deverá contemplar:

- **Socket:** Use uma biblioteca de *socket* para lidar com concorrência e não-bloqueio.
- **Serialização:** Desenvolver um formato para o arquivo de metadados e um protocolo para empacotar/desempacotar as mensagens de bloco.
- **Gerenciamento de Blocos:** Manter um registro local (ex: um array booleano ou hash map) para rastrear quais blocos do arquivo o Peer possui.
- **Montagem do Arquivo:** Após todos os blocos serem recebidos e verificados, remontar e salvar o arquivo final no sistema local.

Abaixo os parâmetros para os testes:

Tabela 1: Parâmetros de Configuração Base para os Testes

Parâmetro	**Valor Padrão**	**Variações de Teste**	**Objetivo da Variação**
Quantidade de Peers	2	4	Testar o impacto dos peers.
Tamanho do Bloco	1024 Bytes (1 KB)	4096 Bytes (4 KB)	Testar o impacto da fragmentação.
Arquivo Pequeno (File A)	10 KB	20 KB	Validar transferência rápida de poucos blocos.
Arquivo Médio (File B)	1 MB	5 MB	Validar fragmentação em número razoável de blocos.
Arquivo Grande (File C)	10 MB	20 MB	Testar estabilidade para grandes transferências.
Configuração de Vizinhos	Estática	N/A	Focar apenas no protocolo P2P.

Para cada teste, seguir os passos:

- **Preparação:** Criar o arquivo de teste (File A, B ou C) com conteúdo aleatório/único e garantir que o *Seeder* inicial tenha o arquivo e o metadado.
- **Início:** Iniciar todos os processos (ex: Peers A, B, C, D) com as configurações de vizinhos especificadas.
- **Monitoramento**: Registrar as mensagens trocadas e o status de blocos de cada Peer.
- **Verificação:**
  - ***Integridade:*** Calcular e comparar o checksum (ex: hash SHA-256 ou similar) do arquivo baixado com o original.
  - ***Remontagem:*** Verificar se o arquivo remontado possui o tamanho correto.
  - ***Log***: Confirmar nos logs que os blocos foram recebidos das fontes esperadas (especialmente nos testes com tamanhos maiores).